TESTAREA SISTEMELOR SOFTWARE · PROIECT DE LABORATOR

Smart Parking Gate

Testare funcțională, structurală și prin mutații pentru motorul de decizie al unei parcări corporate multi-tenant

78
teste automate

V(G) = 7
complexitate ciclomatică

5
teste MC/DC (N + 1)

2
unelte de mutație

Victor Mura — 50%
Cerințele 1, 2, 6 · testare funcțională, acoperire, mutanți manuali

Natalia Moroz — 50%
Cerințele 3, 4, 5, 7 · CFG, MC/DC, mutant echivalent, unelte

Universitatea din București · FMI · Master Ingineria Sistemelor Software, anul II · 2026

STRUCTURA PREZENTĂRII

Echipa

7 cerințe, două perspective, o singură funcție

01

Testare funcțională

EP · BVA · Graf cauză-efect

02

Acoperire de cod

pytest-cov, fiecare set separat

03

Graf de control

CFG · V(G) · căi independente

04

MC/DC

decizia compusă EV, 5 teste

05

Mutant echivalent

ordin 1, demonstrație pe $\mathbb{Z}$

06

Mutanți ne-echivalenți

unul ucis, unul supraviețuitor

07

Unelte de mutație

mutmut vs. cosmic-ray

Victor Mura — 50%

Natalia Moroz — 50%

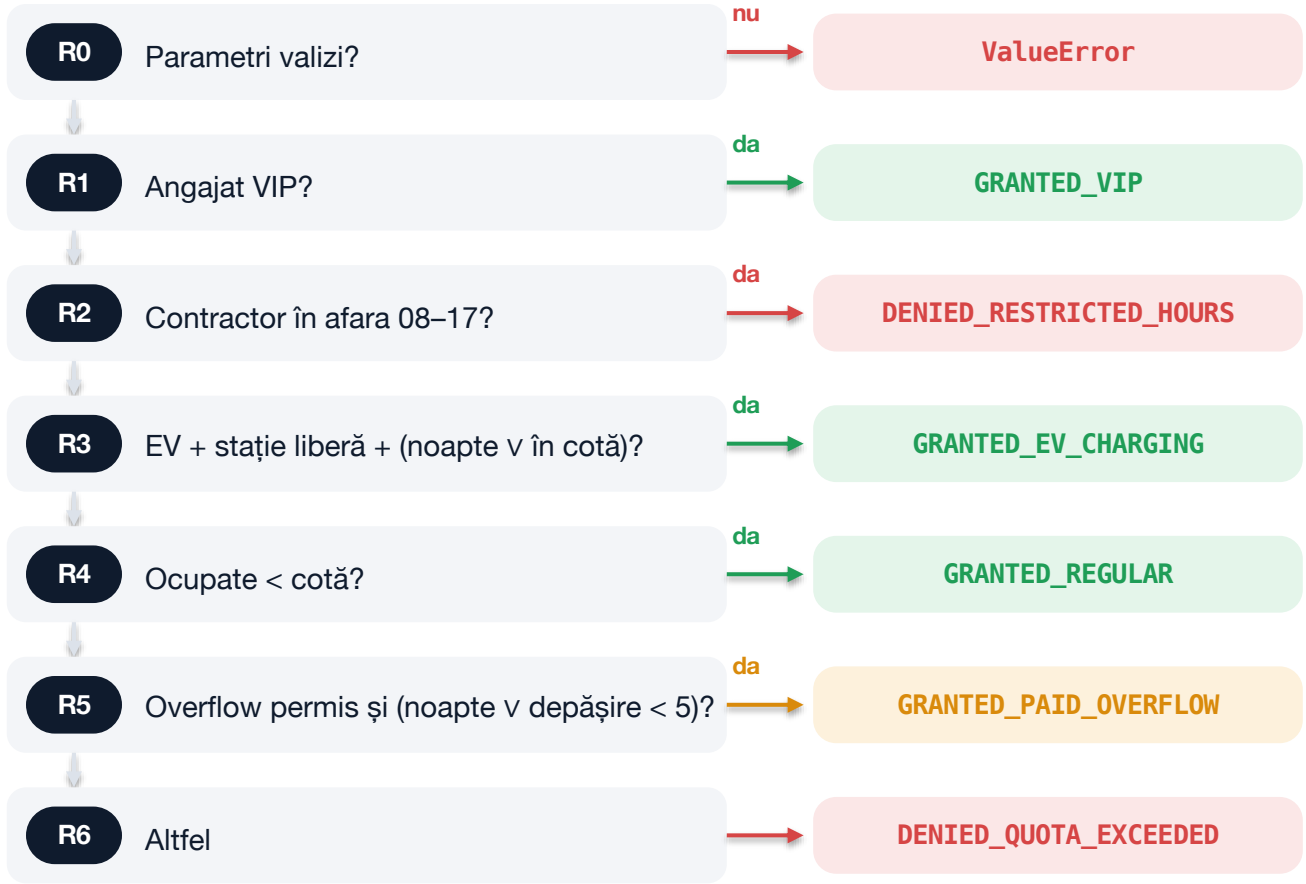
Culoarea fiecărui slide indică autorul secțiunii.

Bariera unui campus de birouri cu mai mulți chiriași

SCENARIU

- **Mai multe companii** împart aceeași parcare; fiecare are o cotă contractuală de locuri gratuite.
- **O singură barieră** decide instant accesul pe baza a 7 parametri.
- **Resurse comune:** stații de încărcare EV și o zonă de overflow plătită de companie.
- **Funcție pură, deterministă** — fără I/O, ideală pentru testare unitară și mutații.

IERARHIA DECIZIILOR — evaluate_parking_access()



Funcția sub test și domeniile de intrare

```
def evaluate_parking_access(quota, occupied, hour, tier,
        is_ev, has_ev_spots, allows_overflow):
    if quota <= 0 or occupied < 0 or hour < 0 or hour > 23:
        raise ValueError(...)
    is_peak = (8 <= hour <= 17)
    if tier == VIP:
        return GRANTED_VIP
    if tier == CONTRACTOR and not is_peak:
        return DENIED_RESTRICTED_HOURS
    within = occupied < quota
    if is_ev and has_ev_spots and (not is_peak or within):
        return GRANTED_EV_CHARGING
    if within:
        return GRANTED_REGULAR
    overflow = occupied - quota
    if allows_overflow and (not is_peak or overflow < 5):
        return GRANTED_PAID_OVERFLOW
    return DENIED_QUOTA_EXCEEDED
```

Parametru	Tip	Domeniu valid
`company_allocated_quota`	int	≥ 1
`company_current_occupied`	int	≥ 0
`entry_hour`	int	0 … 23 (vârf: 8 … 17)
`tier`	Enum	REGULAR · VIP · CONTRACTOR
`is_electric_vehicle`	bool	True / False
`has_ev_spots_available`	bool	True / False
`company_allows_overflow`	bool	True / False

6 rezultate posibile

VIP | EV | REGULAR | OVERFLOW | HOURS ✕ | QUOTA ✕

Simplificat pentru lizibilitate — sursa completă: src/parking_gate.py. Linia evidențiată = decizia MC/DC.

PARTEA I · VICTOR MURA

Testare funcțională

Cerința 1	Partiționare în clase de echivalență, valori de frontieră, graf cauză-efect
Cerința 2	Acoperirea de cod obținută de fiecare set, măsurată separat

Victor Mura

Partiționare în clase de echivalență

Parametru	Clase valide	Clase invalide
quota (q)	q ≥ 1	q = 0 · q < 0
occupied (o)	o < q (sub cotă) · o = q (la cotă) · o > q (peste cotă)	o < 0
entry_hour (h)	0–7 (dimineață) · 8–17 (vârf) · 18–23 (seară)	h < 0 · h ≥ 24
tier	VIP · REGULAR · CONTRACTOR	— (garantat de Enum)
is_ev, has_spots	(T, T) · (T, F) · (F, –)	—
allows_overflow	True · False	—

13

cazuri de test

6 pentru clasele invalide (ValueError).

7 pentru clasele valide, câte unul pentru fiecare rezultat al funcției.

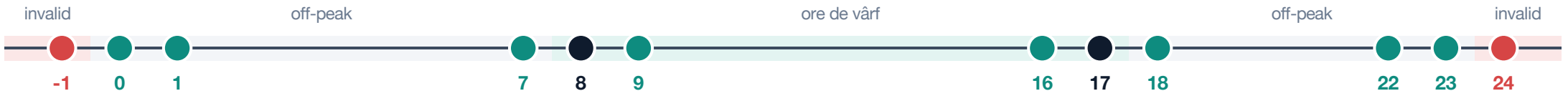
tests/test_ep.py

Ideea: un reprezentant per clasă; dacă funcționează pentru el, presupunem că funcționează pentru toată clasa. Limita: nu spune nimic despre marginile clasei — de aceea urmează BVA.

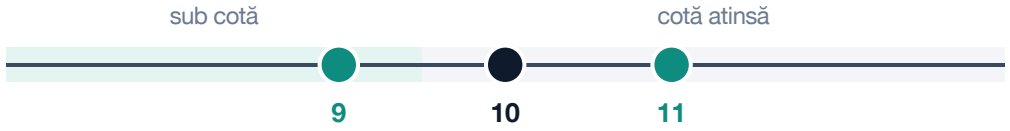
Victor Mura

Analiza valorilor de frontieră

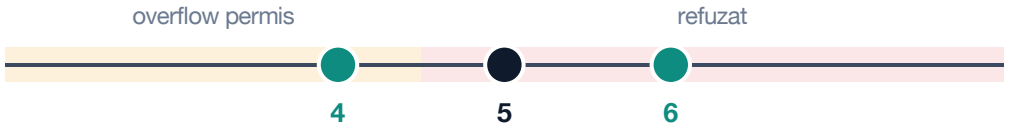
entry_hour — limitele zilei și ale intervalului de vârf



occupied față de quota = 10



depășire la ore de vârf (occupied – quota)



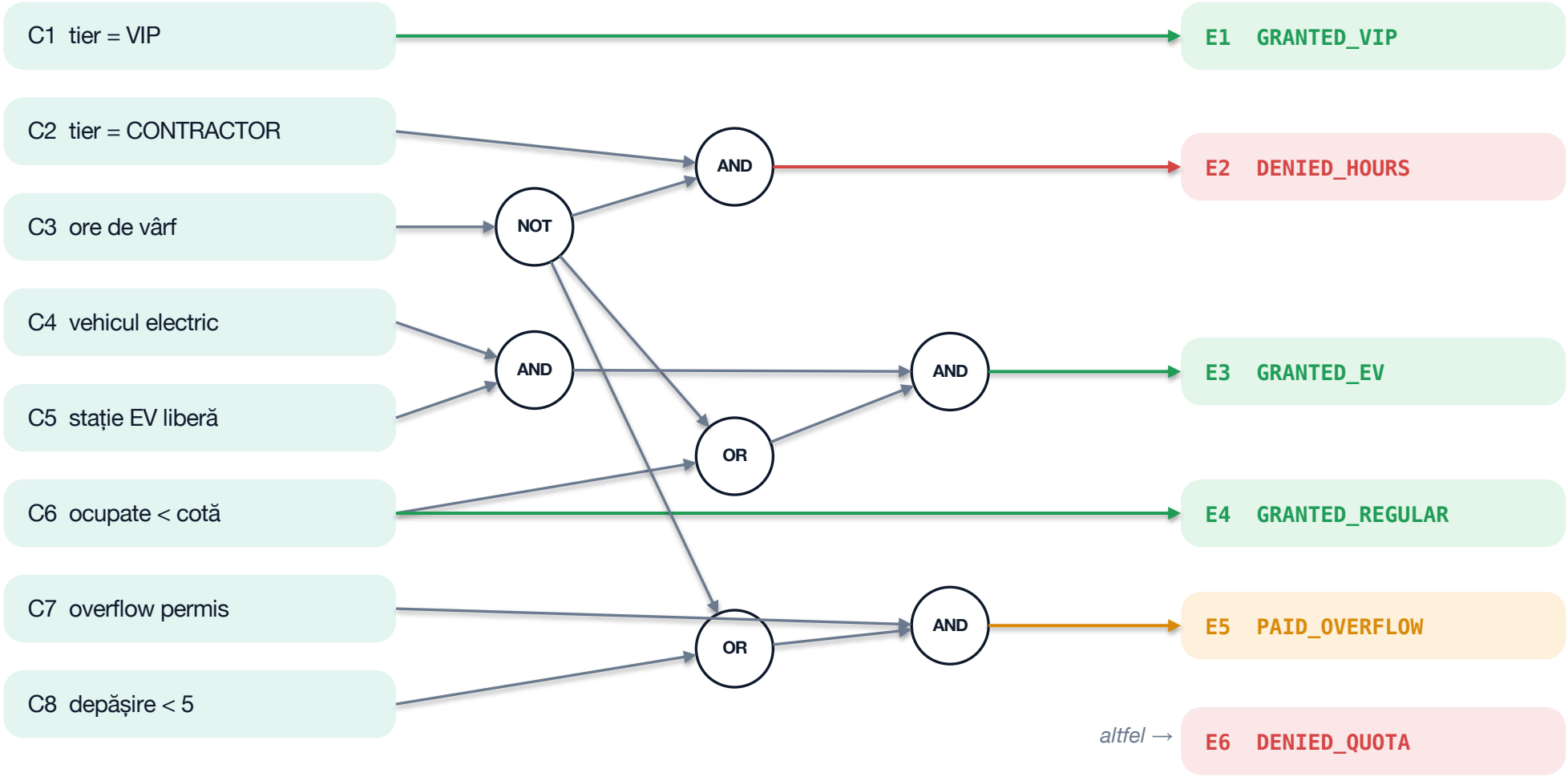
Frontieră	Valori testate	Ce prinde
quota ≥ 1	`0 · 1 · 2`	cote zero sau negative
occupied ≥ 0	`−1 · 0 · 1`	contoare negative
occupied < quota	`9 · 10 · 11`	< scris greșit ca ≤
overflow < 5	`4 · 5 · 6`	pragul strict de overflow

24

cazuri de test
tests/test_bva.py

Victor Mura

Graful cauză-efect



Constrângere

E(C1, C2): un angajat nu poate fi VIP și contractor.

Porți

NOT = ore off-peak
AND/OR = condiții compuse

Prioritate

E1 > E2 > E3 > E4 > E5 > E6

Victor Mura

Din graf în tabel de decizie: 9 reguli → 9 teste

Regulă	C1	C2	C3	C4	C5	C6	C7	C8	Efect
R1	1	0	–	–	–	–	–	–	**E1 VIP**
R2	0	1	0	–	–	–	–	–	**E2 HOURS ✕**
R3	0	0	0	1	1	–	–	–	**E3 EV**
R4	0	0	1	1	1	1	–	–	**E3 EV**
R5	0	0	–	0	–	1	–	–	**E4 REGULAR**
R6	0	0	0	0	–	0	1	–	**E5 OVERFLOW**
R7	0	0	1	0	–	0	1	1	**E5 OVERFLOW**
R8	0	0	1	0	–	0	1	0	**E6 QUOTA ✕**
R9	0	0	–	0	–	0	0	–	**E6 QUOTA ✕**

C1 VIP · C2 contractor · C3 vârf · C4 EV · C5 stație liberă · C6 în cotă · C7 overflow permis · C8 depășire < 5

Cum citim tabelul

1 = cauza trebuie să fie adevărată

0 = cauza trebuie să fie falsă

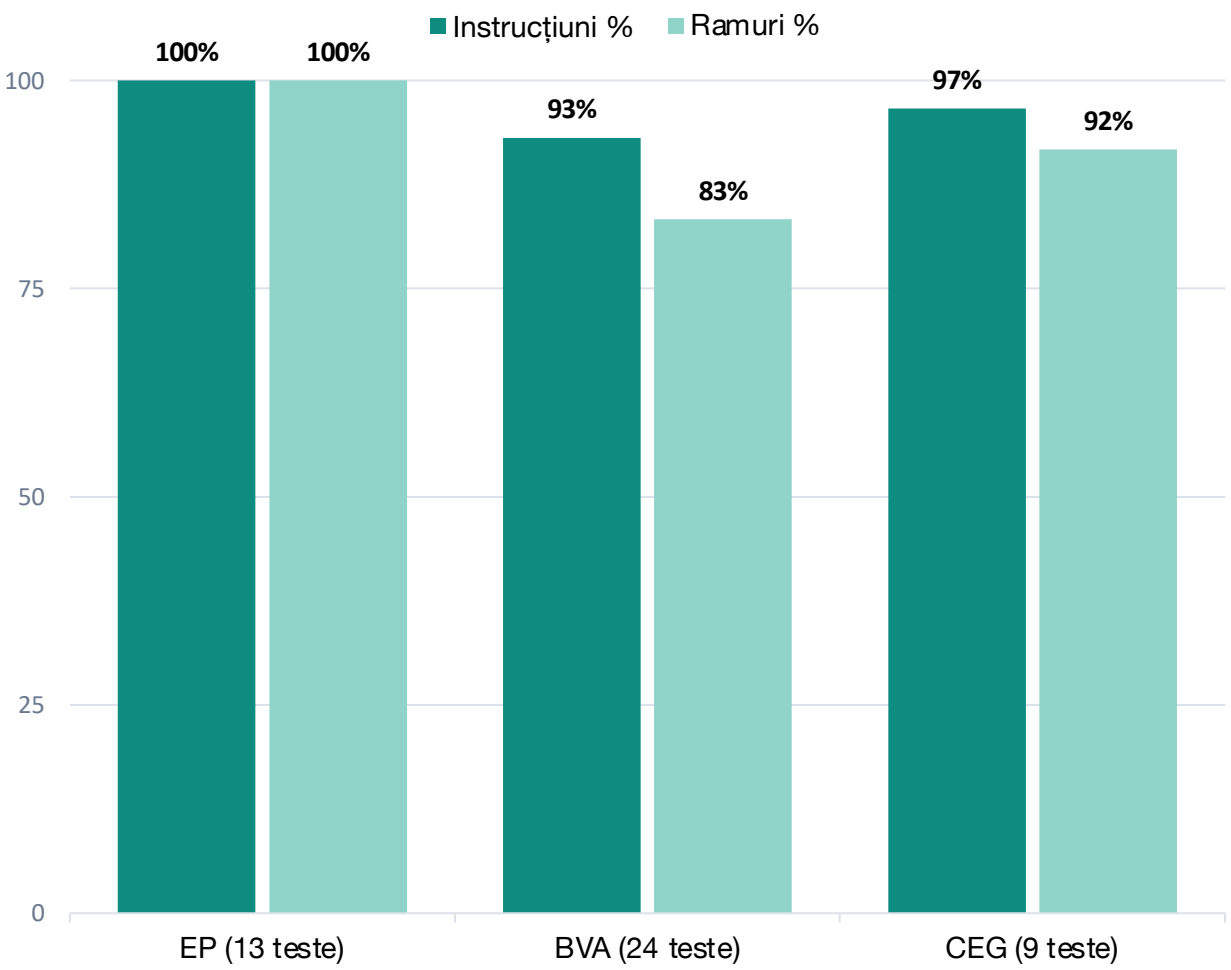
– = nu contează

Fiecare regulă devine exact un caz de test în tests/test_ceg.py.

Cel mai mic set funcțional (9 teste), dar acoperă toate combinațiile logice ale specificației.

Victor Mura

Același cod, trei seturi de teste, trei rezultate



pytest --cov=src --cov-branch, rulat separat pe fiecare set · src/parking_gate.py

EP — 100%

`29/29 instrucțiuni, 12/12 ramuri`

O clasă per rezultat ⇒ fiecare ramură e atinsă cel puțin o dată.

BVA — 90%

`lipsesc liniile 61 și 70`

VIP și EV sunt decizii calitative — nu au „frontieră” numerică de testat.

CEG — 95%

`lipsește linia 54`

Graful modelează doar intrări valide; ValueError nu e o cauză de business.

PARTEA II · NATALIA MOROZ

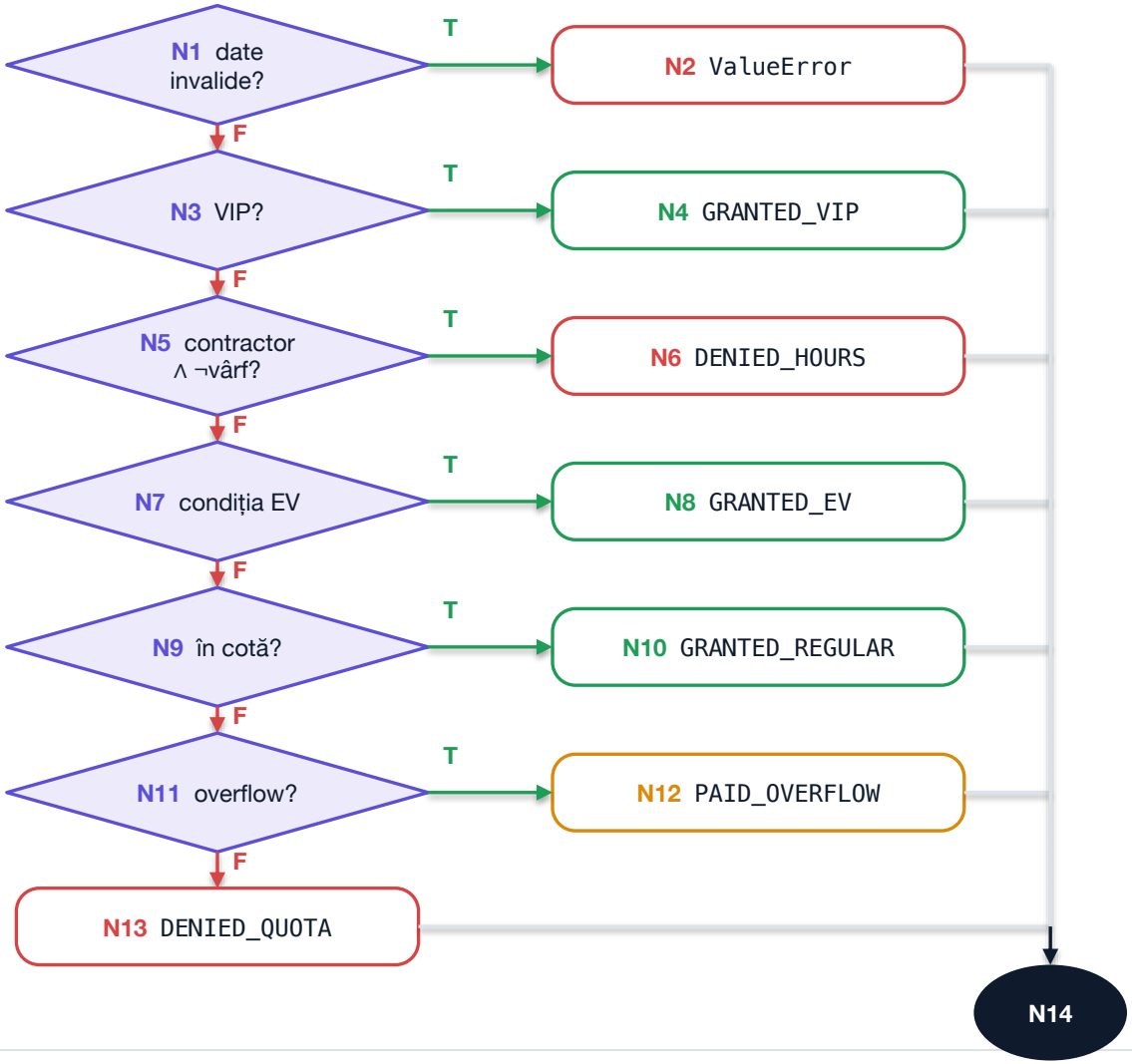
Testare structurală

Cerința 3 Graful de control, complexitatea ciclomatică, căile independente

Cerința 4 Modified Condition / Decision Coverage pe decizia compusă EV

Natalia Moroz

Graful de flux de control



COMPLEXITATE CICLOMATICĂ

V(G) = 7

14 noduri
19 arce
6 noduri predicat

Arce și noduri

E − N + 2P

19 − 14 + 2 = 7

Noduri predicat

P + 1

6 + 1 = 7

Regiuni

R

6 închise + 1 exterioară = 7

Trei metode, același rezultat ⇒ 7 căi liniar independente de acoperit.

Natalia Moroz

7 căi liniar independente, fiecare cu testul ei

Cale	Secvență de noduri	Date de test	Rezultat
P1	N1 → N2	quota = 0	ValueError
P2	N1 → N3 → N4	tier = VIP	GRANTED_VIP
P3	N1 → N3 → N5 → N6	CONTRACTOR, ora 21	DENIED_RESTRICTED_HOURS
P4	… → N5 → N7 → N8	EV, stație liberă, ora 6	GRANTED_EV_CHARGING
P5	… → N7 → N9 → N10	occupied 5 / quota 10	GRANTED_REGULAR
P6	… → N9 → N11 → N12	12 / 10, overflow, ora 10	GRANTED_PAID_OVERFLOW
P7	… → N9 → N11 → N13	10 / 10, fără overflow	DENIED_QUOTA_EXCEEDED

De ce 7? Fiecare cale adaugă cel puțin un arc nou. Împreună acoperă toate cele 19 arce ale grafului, iar orice altă cale prin program se poate scrie ca o combinație a acestora.

Natalia Moroz

MC/DC pe decizia de acces la stația EV

$$D = (A \wedge B) \wedge (\neg C \vee D)$$

A vehicul electric **B** stație liberă
C ore de vârf **D** ocupate < cotă

Test	A	B	C	D	Dec.	Justificare independență
T1	1	1	0	0	1	Bază comună (acces EV aprobat)
T2	0	1	0	0	0	A este 0 ⇒ acces refuzat
T3	1	0	0	0	0	B este 0 ⇒ fără stație liberă
T4	1	1	1	0	0	C este 1 ⇒ vârf, ¬C este 0
T5	1	1	1	1	1	D este 1 ⇒ sub cotă, resuscitează OR

De la 16 combinații la 5 teste

Tabelul exhaustiv are $2^4 = 16$ combinații (prezentate în raport). MC/DC demonstrează independența fiecărei variabile atomice cu doar N + 1 = 5 teste selectate riguros, reducând efortul de testare cu 69% și garantând decizia fără redundanță.

PERECHI DE INDEPENDENȚĂ — se schimbă o singură condiție, se schimbă decizia

A — **T1** 1 1 0 0 → 1 ⟶ **T2** 0 1 0 0 → 0

B — **T1** 1 1 0 0 → 1 ⟶ **T3** 1 0 0 0 → 0

C — **T1** 1 1 0 0 → 1 ⟶ **T4** 1 1 1 0 → 0

D — **T5** 1 1 1 1 → 1 ⟶ **T4** 1 1 1 0 → 0

N = 4 condiții ⇒ N + 1 = 5 teste · tests/test_mcdc.py · 5/5 trec

PARTEA III · ECHIPA

Testare prin mutații

- **Cerința 5** — Un mutant de ordin 1 echivalent — și de ce nu poate fi ucis
- **Cerința 6** — Doi mutanți ne-echivalenți: unul ucis, unul supraviețuitor
- **Cerința 7** — Generare automată cu mutmut și cosmic-ray

Natalia Moroz

Mutant de ordin 1 echivalent

```
# src/parking_gate.py, linia 57
- is_peak_hours = (8 <= entry_hour <= 17)
+ is_peak_hours = (7 < entry_hour < 18)
```

DEMONSTRAȚIE

1. entry_hour este de tip int, deci $h \in \mathbb{Z}$.

2. Pe $\mathbb{Z}$: $h \geq 8 \Leftrightarrow h > 7$ și $h \leq 17 \Leftrightarrow h < 18$.

3. Deci is_peak_hours are aceeași valoare pentru orice h, iar restul codului e identic.

∴ **M(x) = P(x) pentru orice intrare x — niciun test nu poate ucide mutantul.**

VERIFICARE EXHAUSTIVĂ

1 152

comparații original vs. mutant, toate identice
24 ore × 3 roluri × 2^3 combinații bool × 2 niveluri de ocupare

De ce contează

Un mutant echivalent rămâne „viu” oricât de bune ar fi testele.

Scorul de mutație brut îl numără totuși ca eșec — de aceea mutanții echivalenți se exclud din numitor.

Victor Mura

Același test, doi mutanți, două destine

TESTUL ALES (din BVA)

`test_bva_occupied_exactly_at_quota`

`quota = 10 · occupied = 10 · ora 12 · REGULAR fără EV · allows_overflow = False`

Mutant #1 — UCIS

ROR: < devine ≤

```
- within = occupied < quota
+ within = occupied <= quota
```

- 10 <= 10 este True
- funcția returnează GRANTED_REGULAR
- testul așteaptă DENIED_QUOTA_EXCEEDED
- **aserțiunea pică ⇒ mutant detectat**

Mutant #2 — SUPRAVIEȚUIEȘTE

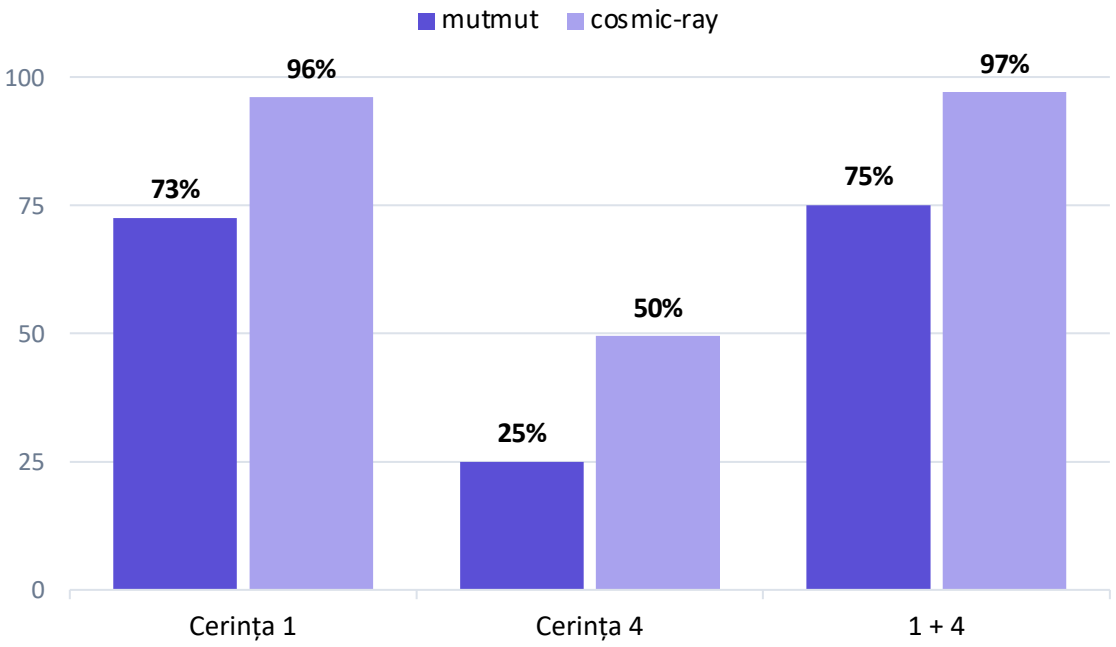
constantă: 5 devine 4

```
- ... (not is_peak or overflow < 5)
+ ... (not is_peak or overflow < 4)
```

- allows_overflow = False
- `False and …` se scurtcircuitează
- pragul mutat nu este evaluat niciodată
- **rezultat identic ⇒ testul nu vede mutația**

Natalia Moroz

mutmut vs. cosmic-ray pe trei seturi de teste



Set de teste	mutmut ucis / total	cosmic-ray ucis / total
Cerința 1	29 / 40	97 / 101
Cerința 4	10 / 40	50 / 101
1 + 4	30 / 40	98 / 101

Operatori diferiți

mutmut generează 40 de mutanți (înlocuiri de literali, enum-uri și operatori relaționali). cosmic-ray generează 101 mutanți axați pe rotația completă a operatorilor de comparație (Lt_Eq, LtE_Lt), operatorilor booleeni (And_Or) și mutarea numerelor.

Ce supraviețuiește

La mutmut, cei 10 mutanți supraviețuitori în setul combinat sunt declarațiile Enum (1-9) și textul mesajului ValueError (19). La cosmic-ray, doar 3 mutanți din 101 supraviețuiesc (97.03% scor de mutație!).

Cerința 1 vs. Cerința 4

Setul funcțional (Cerința 1) oferă o acoperire largă a tuturor ramurilor (72.5% mutmut / 96.0% cosmic-ray). Setul MC/DC (Cerința 4) este hiperspecializat pe decizia compusă EV (ucide 100% din mutanții acesteia). Combinate, cele două seturi ucid practic toți mutanții semantici.

Ce am obținut și ce am învățat

78
teste, toate trec

100%
ramuri acoperite (EP)

7
căi independente

75%
scor mutmut, set combinat

Tehnicile se completează

BVA prinde erorile off-by-one, CEG combinațiile logice, EP domeniile uitate.

Acoperirea nu e suficientă

100% ramuri nu garantează că un < nu a devenit ≤ — mutațiile o arată.

Scurtcircuitarea ascunde defecte

O condiție evaluată leneș poate face un test „orb” la o mutație.

Utilizare AI

Asistentul Antigravity a fost folosit pentru schelet de cod, teste și documentație.

Prompturile, rezultatele și verificările sunt documentate în AI_USAGE_LAB.md, conform cerinței.

Cod, teste, documentație: `github.com/victormura/tss` · **Întrebări?**